End-to-End Compilation Path for SYCL to ARM SVE

Andreas Sinanis

1 Motivation

The performance of AI and neural networks is bottlenecked by the execution speed of large matrix multiplications. To speed up this process, we introduce a custom compiler pipeline that automatically restructures these multiplications from nested loops into vector instructions.

2 Tools and Architecture

The SYCL programming model provides mechanisms to perform computations on heterogeneous processing elements, such as CPU cores or GPU threads. Two fundamental constructs for defining this execution are single_task[1] and parallel_for[2].

The `parallel_for` construct is designed for data-parallel execution and distributes iterations of a workload across multiple processing elements, allowing developers to easily express parallelism, managing the parallel execution autonomously. In contrast, `single_task` defines a unit of work that is executed sequentially by a single work-item on the target device.

While parallel_for is the standard approach for SYCL performance, it presents a significant obstacle for custom compiler optimization pipelines. Because parallel_for abstracts the loop distribution into the SYCL runtime, it hides the explicit loop boundaries and data dependency graphs from the LLVM middle-end optimizers. For this thesis, we replace the standard parallel_for implementations with single_task constructs containing explicit, nested C++ for loops. By passing deeply nested loops into the compiler pipeline, we provide the Polly optimizer with a clear preview of the algorithm. Because Polly can see the full structure, it can calculate exactly how memory is being accessed. This allows Polly to automatically reorder the code to ensure data is read in a continuous straight line. Once Polly organizes the memory access, the LLVM Loop Vectorizer converts the optimized loops into Scalable Vector Extension (SVE) instructions.

[1] Intel SYCL single_task documentation: `https://www.intel.com/content/www/us/en/docs/sycl/introduction/latest/02-sycl-basic-code-single-task.html`

[2] Intel SYCL parallel_for documentation: `https://www.intel.com/content/www/us/en/docs/sycl/introduction/latest/03-sycl-basic-code-parallel-for.html`

3 Methodology

To isolate the effects of different compiler optimizations, we designed a custom compilation pipeline and manually drove the LLVM toolchain through three paths:

Path 0 (Baseline): The SYCL code is compiled into standard LLVM IR with all vectorization strictly disabled (-fno-vectorize). This establishes our scalar execution floor.

Path 1 (Vectorized - SVE): The LLVM IR is passed through the opt middle-end with the LLVM Loop Vectorizer enabled (-passes='loop-vectorize') and the target architecture is explicitly set to Neoverse-V2 to trigger Scalable Vector Extension (SVE) instruction generation.

Path 2 (Polyhedral + Vectorized): Before vectorization, the Clang frontend applies the Polly framework (-mllvm -polly). Polly restructures the memory access patterns of the loops and then the output is fed into the LLVM Loop Vectorizer.

Applying this pipeline directly to SYCL parallel_for constructs yielded no vectorization benefits; therefore, the methodology required a manual code transformation. We stripped the parallel_for dispatchers and rewrote the matrix operations into for loops inside a single_task kernel.

4 Setup and Benchmarks

To evaluate the proposed compilation pipeline, experiments were performed using a custom LLVM-18 toolchain targeting the ARM Neoverse-V2. This pipeline used the Intel LLVM SYCL compiler to extract Intermediate Representation (IR) code and ensure compatibility between the SYCL frontend and the standard LLVM middle-end.

All compilations were performed at the -O3 optimization level, and default frontend vectorization was explicitly disabled with -fno-vectorize and -fno-slp-vectorize to maintain control over the vectorization passes in the opt middle-end. For the SVE and Polly paths, the LLVM loop-vectorize pass was explicitly invoked alongside the -mcpu=neoverse-v2 and -mattr=+sve flags to guarantee the emission of Scalable Vector Extension instructions.

To evaluate the compilation pipeline across different memory access patterns, we selected five distinct kernels from the Polybench suite: GEMM, SYRK, and 2MM to represent $\mathcal{O}(N^3)$ complexity, along with 2D-Convolution and BiCG to represent $\mathcal{O}(N^2)$ complexity.

4.1 Code Transformations

Before processing these benchmarks through our custom LLVM pipeline, the original SYCL implementations had to be restructured. Because the default Polybench SYCL

code relies on runtime abstractions that cover the mathematical algorithms from static compiler analysis, we applied the following systematic transformations to all benchmarks (using the 2D-Convolution kernel as a primary example):

- **Replacing SYCL Accessors with Raw Pointers:**
  The original code used SYCL's heavy buffer-accessor model (e.g., auto A = A_buffer.get_access¡...¿(cgh)). Instead, we extracted raw C++ pointers (DATA_TYPE* raw_A = A.data();) before submitting the kernel, and passed these raw pointers directly into the lambda function. Since SYCL accessors introduce complex, opaque C++ wrapper classes, stripping these away and using flat 1D raw pointers exposes the unmodified memory addresses directly to the LLVM frontend.

- **Transitioning from parallel_for** to single_task:
  We replaced the cgh.parallel_for dispatch with cgh.single_task because parallel_for hides the loop distribution within the protected SYCL runtime code. By executing the kernel as a single_task, we prevent the SYCL runtime from interfering with the loop structure, reserving the parallelization structure entirely for our LLVM toolchain.

- **Replacing Implicit Indices and Conditionals with Explicit Nested Loops:**
  In the original parallel_for, the iteration space was implicitly defined by item[0] and item[1], requiring an internal if statement (if((i ¿ 0) && (j ¿ 0)...)) to prevent out-of-bounds accesses. For our version, we removed the if statement and wrote explicit C++ nested loops (e.g., for(size_t i = 1; i ¡ size_ - 1; ++i)) to provide the compiler with a serial Iteration Domain.

- **Injecting Explicit Vectorization Directives:**
  Right before the innermost j loop, we injected specific Clang compiler directives: #pragma clang loop vectorize_width(4, scalable) and #pragma clang loop interleave_count(1). While LLVM opt can attempt auto-vectorization, these pragmas explicitly guide the compiler to target the inner loop for Scalable Vector Extension (SVE).

```
void run(std::vector<sycl::event>& events) {
    using namespace sycl;

    events.push_back(args.device_queue.submit([&](handler& cgh) {
      auto A = A_buffer.get_access<access::mode::read>(cgh);
      auto B = B_buffer.get_access<access::mode::discard_write>(cgh);

      cgh.parallel_for<class conv2D>(B_buffer.get_range(), [=, size_ =
   size](item<2> item) {
        const auto i = item[0];
        const auto j = item[1];

```

```
        const DATA_TYPE c11 = +0.2, c21 = +0.5, c31 = -0.8;
        const DATA_TYPE c12 = -0.3, c22 = +0.6, c32 = -0.9;
        const DATA_TYPE c13 = +0.4, c23 = +0.7, c33 = +0.10;

        if((i > 0) && (j > 0) && (i < size_ - 1) && (j < size_ - 1)) {
          B[item] = c11 * A[{(i - 1), (j - 1)}] + c12 * A[{(i + 0), (j - 1)}] + c13 * A[{(i + 1), (j - 1)}] +
                    c21 * A[{(i - 1), (j + 0)}] + c22 * A[{(i + 0), (j + 0)}] + c23 * A[{(i + 1), (j + 0)}] +
                    c31 * A[{(i - 1), (j + 1)}] + c32 * A[{(i + 0), (j + 1)}] + c33 * A[{(i + 1), (j + 1)}];
        }
      });
    }));
  }
```

Listing 1: Original Polybench SYCL `parallel_for` implementation of 2D-Convolution.

```
void run(std::vector<sycl::event>& events) {
    using namespace sycl;
    float* raw_A = A.data();
    float* raw_B = B.data();

    events.push_back(args.device_queue.submit([&](handler& cgh) {

      cgh.single_task<class conv2D>([=, size_ = size]() {

        float* ptrA = raw_A;
        float* ptrB = raw_B;
        const float c11 = +0.2f, c21 = +0.5f, c31 = -0.8f;
        const float c12 = -0.3f, c22 = +0.6f, c32 = -0.9f;
        const float c13 = +0.4f, c23 = +0.7f, c33 = +0.10f;

        for(size_t i = 1; i < size_ - 1; ++i) {

          #pragma clang loop vectorize_width(4, scalable)
          #pragma clang loop interleave_count(1)
          for(size_t j = 1; j < size_ - 1; ++j) {

            ptrB[i * size_ + j] =
                c11 * ptrA[(i - 1) * size_ + (j - 1)] + c12 * ptrA[(i + 0) * size_ + (j - 1)] + c13 * ptrA[(i + 1) * size_ + (j - 1)] +
                c21 * ptrA[(i - 1) * size_ + (j + 0)] + c22 * ptrA[(i + 0) * size_ + (j + 0)] + c23 * ptrA[(i + 1) * size_ + (j + 0)] +
                c31 * ptrA[(i - 1) * size_ + (j + 1)] + c32 * ptrA[(i + 0) * size_ + (j + 1)] + c33 * ptrA[(i + 1) * size_ + (j + 1)];
          }
        }
      });
    }));
```

```
30   }
```

Listing 2: Transformed SYCL `single_task` implementation using raw pointers, explicit loops, and vectorization pragmas.

4.2 Static Code Analysis

Before analyzing the execution time improvements, it is necessary to verify that our custom compilation pipeline successfully applied the intended transformations. Because SYCL abstracts hardware execution, we examined the generated LLVM IR prior to back-end lowering to confirm that vectorization was successfully applied.

4.2.1 Verification of Vectorization for SVE Path

To confirm the successful emission of vector instructions, we compared the inner loop of the 2D-Convolution kernel between the baseline scalar compilation and the LLVM Loop Vectorizer pass.

In the Baseline, the compiler generated standard scalar instructions operating on single 32-bit floating-point values. As shown in Listing 3, the memory controller loads a single float, and the math is handled by the scalar @llvm.fmuladd.f32 intrinsic.

```
for.body7.i:
  %sub10.i = add i64 %j.0.i, -1
  %arrayidx.i = getelementptr float, ptr addrspace(1) %0, i64 %sub10.i
  %3 = load float, ptr addrspace(1) %arrayidx.i, align 4, !tbaa !15
  %arrayidx17.i = getelementptr float, ptr addrspace(1) %1, i64 %sub10.
   i
  %4 = load float, ptr addrspace(1) %arrayidx17.i, align 4, !tbaa !15
  %mul18.i = fmul float %4, 0xBFD3333340000000
  %5 = tail call float @llvm.fmuladd.f32(float %3, float 0
   x3FC99999A0000000, float %mul18.i)
```

Listing 3: Baseline LLVM IR showing scalar loads and intrinsics.

By contrast, successfully captured the explicit #pragma clang loop directives we injected into the single_task C++ code, the LLVM middle-end replaced the scalar loads and mathematical intrinsics with vectorized intrinsics, as demonstrated in Listing 4.

```
vector.body:
  %index = phi i64 [ 0, %vector.ph ], [ %index.next, %vector.body ]
  %vec.ind = phi <4 x i64> [ <i64 1, i64 2, i64 3, i64 4>, %vector.ph
   ], [ %vec.ind.next, %vector.body ]
  %offset.idx = add i64 1, %index
  %18 = add i64 %offset.idx, 0
  %19 = add i64 %18, -1
  %20 = getelementptr float, ptr addrspace(1) %13, i64 %19
  %21 = getelementptr float, ptr addrspace(1) %20, i32 0
  %wide.load = load <4 x float>, ptr addrspace(1) %21, align 4, !tbaa
   !13, !alias.scope !15
  %22 = getelementptr float, ptr addrspace(1) %14, i64 %19
```

```
  %23 = getelementptr float, ptr addrspace(1) %22, i32 0
  %wide.load15 = load <4 x float>, ptr addrspace(1) %23, align 4, !tbaa
    !13, !alias.scope !18
  %24 = fmul <4 x float> %wide.load15, <float 0xBFD3333340000000, float
    0xBFD3333340000000, float 0xBFD3333340000000, float 0
    xBFD3333340000000>
  %25 = call <4 x float> @llvm.fmuladd.v4f32(<4 x float> %wide.load, <4
    x float> <float 0x3FC99999A0000000, float 0x3FC99999A0000000, float
    0x3FC99999A0000000, float 0x3FC99999A0000000>, <4 x float> %24)
```

Listing 4: Vectorized LLVM IR (Path 1) showing 128-bit vector loads and intrinsics.

Analysis

Listing 4 provides static proof that the pipeline successfully parallelized the IR. The vectorized instruction `load <4 x float>` indicates that the memory controller is now fetching 128-bit chunks of contiguous memory, four 32-bit floats simultaneously. Furthermore, the scalar @llvm.fmuladd.f32 intrinsic has been automatically upgraded to the vectorized @llvm.fmuladd.v4f32 intrinsic. This confirms that the LLVM backend will successfully map these operations directly to ARM's Scalable Vector Extension (SVE) fused multiply-add hardware, reducing the total number of instructions per loop iteration.

4.3 Verification of Poly Vectorization

While SVE path successfully vectorized the operations, complex stencil access patterns (such as the 3×3 grid in the `2D-Convolution` kernel) introduce severe memory aliasing concerns. Without advanced static analysis, the compiler cannot guarantee that reading from array `A` and writing to array `B` will not overlap in memory, which often prevents optimal vectorization or forces the compiler to insert expensive runtime bounds-checking.

To verify Polly vectorization impact, we examined the generated LLVM IR shown in Listing 5.

```
polly.stmt.for.body7.i:
  %46 = add i64 %38, %45
  %scevgep = getelementptr i8, ptr addrspace(1) %_arg_raw_A, i64 %46
  %_p_scalar_ = load float, ptr addrspace(1) %scevgep, align 4, !alias.
    scope !19, !noalias !22
```

Listing 5: Polly-Optimized LLVM IR showing SCEV pointer arithmetic and strict memory aliasing metadata.

Analysis of Polyhedral Restructuring

Listing 5 illustrates the advantages of the Polyhedral model, First, Polly utilizes Scalar Evolution, the %scevgep instruction, to calculate exact memory access patterns and hoist complex index arithmetic out of the innermost loop. Second, Polly injects strict metadata tags !alias.scope and !noalias into every memory load. Because Polly models the iteration domain of the single_task loop and statically proves to the LLVM backend that the memory regions for matrix A and matrix B will not overlap.

By providing this information, Polly allows the LLVM Loop Vectorizer to perform vectorized intrinsics. As shown in Listing 6, the final output of Polly Path combines Polly memory safety with SVE vectorization.

```
vector.body:
  %52 = getelementptr i8, ptr addrspace(1) %_arg_raw_A, i64 %51
  %53 = getelementptr float, ptr addrspace(1) %52, i32 0
  %wide.load = load <4 x float>, ptr addrspace(1) %53, align 4, !alias.
   scope !19, !noalias !22
  %54 = add i64 %45, %50
  %55 = getelementptr i8, ptr addrspace(1) %_arg_raw_A, i64 %54
  %56 = getelementptr float, ptr addrspace(1) %55, i32 0
  %wide.load2 = load <4 x float>, ptr addrspace(1) %56, align 4, !alias
   .scope !19, !noalias !22
  %57 = fmul <4 x float> %wide.load2, <float 0xBFD3333340000000, float
   0xBFD3333340000000, float 0xBFD3333340000000, float 0
   xBFD3333340000000>

```

Listing 6: Polly IR with memory safety and 128-bit scalable vectorization.

As seen in Listing 6, the vectorizer successfully emits contiguous 128-bit vector loads %wide.load fused with the !noalias.

4.4 Backend Lowering, Assembly Analysis

To observe the final hardware impact of our compilation paths, we analyzed the lowered assembly .s files generated by the LLVM backend, where the assembly reflects the advantages of Polly optimizations and SVE optimization established in the IR analysis.

4.4.1 The Cost of Runtime Memory Checks

Because SVE Path cannot statically guarantee that the memory addresses of array A and array B will not overlap, the LLVM backend has to insert runtime bounds to check every block before entering the vectorized loop, as shown in the listing 7. The backend generates a vector.memcheck block with cmp and ccmp (conditional compare) instructions.

```
.LBB0_5:  // %vector.memcheck
  cmp   x19, x0
  ccmp  x30, x26, #2, lo
  cset  w5, lo
  cmp   x19, x7
  ccmp  x2, x26, #2, lo
  cset  w7, lo
  b.lo  .LBB0_14
```

Listing 7: SVE Path AArch64 Assembly showing expensive runtime memory overlap checks.

This block forces the CPU to evaluate pointers during execution, wasting clock cycles. If the hardware detects any overlap, it aborts vectorization and branches b.lo to a scalar

loop.

4.4.2 Polly Alias Address Elimination

Because Polly proved memory safety via the !noalias tags, the compiler eliminated the vector.memcheck block from the final assembly and the CPU can enter the vectorized loop without wasting cycles on pointer validation.

```
.LBB0_20:
  ldr   q18, [x23], #16
  fmla  v19.4s, v7.4s, v18.4s
  ldr   q18, [x18], #16
  fmla  v19.4s, v16.4s, v18.4s
  str   q19, [x15], #16
```

Listing 8: Assembly showing Polly contiguous vector loads.

As demonstrated in Listing 8, the compiler utilizes 128-bit vector registers q and v registers and the instruction ldr q18, [x23], #16 loads four 32-bit floats simultaneously and automatically increments the pointer register x23 by 16 bytes.

5 Evaluation

In this section, we analyze the execution behavior of the five selected SYCL benchmarks 2D-Convolution, 2MM, BiCG, GEMM, and SYRK across our custom LLVM pipeline. To evaluate the efficacy of the Polyhedral model versus standard SVE passes, we isolated the nested loops for each kernel.

5.1 SVE Efficacy

Initial execution time benchmarks revealed a contrast in how different algorithmic complexities responded to the optimization paths. Of the five evaluated snippets, 2D-Convolution kernel demonstrated major speedups under both the SVE and Polly optimization paths, but for the remaining kernels 2MM, BiCG, GEMM, and SYRK, performance improvements were observed only under the SVE optimization path.

5.2 Vectorization Bottlenecks in GEMM

In original configuration of GEMM, the LLVM Loop Vectorizer failed to emit efficient SVE instructions. As shown in Listing 9, the innermost k loop iterates over the columns of ptrA and the rows of ptrB, while repeatedly updating a single scalar location in ptrC:

```
for(size_t i = 0; i < NI_; i++) {
  for(size_t j = 0; j < NJ_; j++) {
    ptrC[i * NJ_ + j] *= BETA;
    #pragma clang loop vectorize_width(4, scalable)
```

```
    #pragma clang loop interleave_count(1)
    for(size_t k = 0; k < NK_; k++) {
      ptrC[i * NJ_ + j] += ALPHA * ptrA[i * NK_ + k] * ptrB[k * NJ_ + j
   ];
    }
  }
}
```

Listing 9: Original GEMM Structure forcing non-contiguous memory instructions.

This access pattern forces the vectorizer to rely on non-contiguous memory instructions. To resolve this we manually applied a loop interchange transformation, swapping the j and k loops and hoisting the loop-invariant reads of matrix A, as demonstrated in Listing 10:

```
for(size_t i = 0; i < NI_; i++) {
  for(size_t j = 0; j < NJ_; j++) {
    ptrC[i * NJ_ + j] *= BETA;
  }
  for(size_t k = 0; k < NK_; k++) {
    float alpha_a_ik = ALPHA * ptrA[i * NK_ + k];
    #pragma clang loop vectorize_width(4, scalable)
    #pragma clang loop interleave_count(1)
    for(size_t j = 0; j < NJ_; j++) {
      ptrC[i * NJ_ + j] += alpha_a_ik * ptrB[k * NJ_ + j];
    }
  }
}
```

Listing 10: Optimized GEMM Structure.z

By making j the innermost loop, both ptrC and ptrB are accessed sequentially. This stride memory access pattern allows the LLVM backend to emit perfectly contiguous SVE load and store instructions.

5.3 SVE Vectorization Verification in the GEMM Kernel

Following the manual loop interchange transformation applied to the GEMM kernel, we analyzed the generated LLVM Intermediate Representation from SVE Path to verify that the compiler successfully capitalized on the stride memory access pattern.

Vector Length Agnostic Instructions for GEMM

To verify the successful SVE compilation is the presence of scalable vector types in the IR, as shown in Listing 11, the memory loads operate on `<vscale x 4 x float>` rather than `<4 x float>` fixed width types.

```
vector.body:
  %wide.load = load <vscale x 4 x float>, ptr addrspace(1) %62, align
   4, !tbaa !13, !alias.scope !21
  %63 = getelementptr float, ptr addrspace(1) %42, i64 %60
```

```
  %64 = getelementptr float, ptr addrspace(1) %63, i32 0
  %wide.load5 = load <vscale x 4 x float>, ptr addrspace(1) %64, align
   4, !tbaa !13, !alias.scope !24, !noalias !21
  %65 = call <vscale x 4 x float> @llvm.fmuladd.nxv4f32(<vscale x 4 x
   float> %broadcast.splat, <vscale x 4 x float> %wide.load, <vscale x
   4 x float> %wide.load5)
  store <vscale x 4 x float> %65, ptr addrspace(1) %64, align 4, !tbaa
   !13, !alias.scope !24, !noalias !21
```

Listing 11: Optimized GEMM IR (Path 1) showing Scalable Vector types and intrinsics.

The use of vscale demonstrates that the LLVM Loop Vectorizer successfully generated Vector Length Agnostic code, moreover, the compiler utilizes the @llvm.fmuladd.nxv4f32 intrinsic, which maps to the SVE fmla, Fused Multiply Add hardware instruction.

Scalar Broadcasting (Splatting) from Loop-Invariant Motion

Because the loop interchange transformation moved the iteration over matrix A ptrA[i * NK_ + k] outside the innermost j loop, the compiler recognized it as loop-invariant data, listing 12 describes how the vectorizer handles this optimization.

```
vector.ph:
%broadcast.splatinsert = insertelement <vscale x 4 x float> poison,
   float %mul16.i, i64 0
  %broadcast.splat = shufflevector <vscale x 4 x float> %broadcast.
   splatinsert, <vscale x 4 x float> poison, <vscale x 4 x i32>
   zeroinitializer
```

Listing 12: GEMM IR - Scalar broadcasting (splatting) of the loop-invariant value.

Instead of redundantly fetching the scalar value from matrix A during every vector iteration, the compiler loads the value once %mul16.i and utilizes insertelement and shufflevector to broadcast this scalar across all lanes of the scalable vector register and ensures the pipeline is not bottlenecked by redundant L1 cache requests.

```
vector.memcheck:
  %bound0 = icmp ult ptr addrspace(1) %scevgep1, %scevgep4
  %bound1 = icmp ult ptr addrspace(1) %scevgep3, %scevgep2
  %found.conflict = and i1 %bound0, %bound1
  br i1 %found.conflict, label %scalar.ph, label %vector.ph
```

Listing 13: GEMM IR - Runtime memory conflict fallback block.

As demostrated in Listing 13, the backend injected a vector.memcheck block, beacue if an overlap is detected during execution %found.conflict, the CPU will have to abandon the SVE vectors and branch to the regular scalar.ph loop.

5.4 SYRK Vectorization and Loop Fusion

Our initial approach to vectorizing the SYRK kernel involved a direct translation of the standard mathematical formulation into SYCL, as shown in Listing 14, this implementa-

tion utilizes two completely separate nested loops: one to scale the matrix by beta, and another to perform the matrix multiplication accumulation.

```
    for(size_t i = 0; i < size_; i++) {
        #pragma clang loop vectorize_width(4, scalable)
        #pragma clang loop interleave_count(1)
        for(size_t j = 0; j < size_; j++) {
          ptrC[i * size_ + j] *= beta;
        }
    }

    for(size_t i = 0; i < size_; i++) {
        for(size_t j = 0; j < size_; j++) {
            #pragma clang loop vectorize_width(4, scalable)
            #pragma clang loop interleave_count(1)
            for(size_t k = 0; k < size_; k++) {
              ptrC[i * size_ + j] += alpha * ptrA[i * size_ + k] * ptrA
    [j * size_ + k];
            }
        }
    }
```

Listing 14: SYCL implementation of the SYRK.

However, early execution profiling indicated that this approach forces redundant memory sweeps.

To empirically prove the inefficiency of this, we analyzed the IR generated by the LLVM, and the resulting IR confirms that the compiler preserves the two separate loop structures, resulting in redundant memory sweeps over Matrix C that bottleneck memory bandwidth.

As shown in Listing 15, the control flow graph emits two distinct loop nests. The first vector loop vector.body fetches elements from memory, applies the β scalar, and writes the data back to RAM. Following, the program exits this loop and enters an separate loop nest for.cond13.i. To perform the α accumulation, the CPU is forced to issue a redundant load instruction in the preheader for.cond24.i.preheader to refetch the exact same elements of Matrix C. After completing the vector reduction, it issues a second, redundant store instruction middle.block10.

```
vector.body:                                      ; preds = %vector.
    body, %vector.ph
  %index = phi i64 [ 0, %vector.ph ], [ %index.next, %vector.body ]
  %20 = add i64 %index, 0
  %21 = getelementptr float, ptr addrspace(1) %7, i64 %20
  %22 = getelementptr float, ptr addrspace(1) %21, i32 0
  %wide.load = load <vscale x 4 x float>, ptr addrspace(1) %22, align
    4, !tbaa !13
  %23 = fmul <vscale x 4 x float> %wide.load, shufflevector (<vscale x
    4 x float> insertelement (<vscale x 4 x float> poison, float
    1.451200e+04, i64 0), <vscale x 4 x float> poison, <vscale x 4 x i32
    > zeroinitializer)
```

```
  store <vscale x 4 x float> %23, ptr addrspace(1) %22, align 4, !tbaa !13
  %index.next = add nuw i64 %index, %19
  %24 = icmp eq i64 %index.next, %n.vec
  br i1 %24, label %middle.block, label %vector.body, !llvm.loop !15

for.cond24.i.preheader:                           ; preds = %for.cond19.i
  %mul35.i = mul i64 %j18.0.i, %_arg_size_
  %36 = getelementptr float, ptr addrspace(1) %_arg_raw_A, i64 %mul35.i
  %arrayidx42.i = getelementptr float, ptr addrspace(1) %30, i64 %j18.0.i
  %arrayidx42.i.promoted = load float, ptr addrspace(1) %arrayidx42.i, align 4, !tbaa !13
  %37 = call i64 @llvm.vscale.i64()
  %38 = mul i64 %37, 4
  %min.iters.check12 = icmp ule i64 %6, %38
  br i1 %min.iters.check12, label %scalar.ph11, label %vector.memcheck

middle.block10:                                   ; preds = %vector.body17
  %54 = call float @llvm.vector.reduce.fadd.nxv4f32(float -0.000000e+00, <vscale x 4 x float> %52)
  store float %54, ptr addrspace(1) %arrayidx42.i, align 4, !tbaa !13
  br label %scalar.ph11
```

Listing 15: LLVM IR of the SYRK implementation highlighting the redundant load and store instructions across separated loop nests.

This double traversal of Matrix C splits the memory bandwidth and L1 cache, therefore to optimize hardware utilization and eliminate redundant load and store instructions, we applied loop fusion for the scaling and accumulation phases into a single execution pass, as demonstrated in Section 5.2. (Listing 16).

```
for(size_t i = 0; i < size_; i++) {
  for(size_t j = 0; j < size_; j++) {
    ptrC[i * size_ + j] *= beta;
    {
      #pragma clang fp reassociate(on)
      #pragma clang loop vectorize_width(4, scalable)
      #pragma clang loop interleave_count(1)
      for(size_t k = 0; k < size_; k++) {
        ptrC[i * size_ + j] += alpha * ptrA[i * size_ + k] * ptrA[j * size_ + k];
      }
    }
  }
}
```

Listing 16: Fused SYRK kernel utilizing direct-memory updates in the innermost loop.

5.4.1 Validating SVE Memory Promotion

We analyzed the IR generated for this version to prove that the LLVM SVE vectorizer optimized the fused direct memory implementation, and despite the source code explicitly writing to memory inside the innermost loop (ptrC[i * size_ + j] += alpha * ptrA[i * size_ + k] * ptrA[j * size_ + k]), following listing17 confirms that the IR due to loop invariant code and memory promotion eliminated these redundant memory accesses entirely.

The compiler recognized that the memory address for ptrC remained unchanged during the innermost k loop and instead of issuing a memory store instruction, the compiler established a scalable vector accumulator in the vector preheader vector.ph, initializing it with the scaled β value %mul13.i.

```
vector.ph:                                        ; preds = %vector.memcheck
  %18 = call i64 @llvm.vscale.i64()
  %19 = mul i64 %18, 4
  %n.mod.vf = urem i64 %3, %19
  %20 = icmp eq i64 %n.mod.vf, 0
  %21 = select i1 %20, i64 %19, i64 %n.mod.vf
  %n.vec = sub i64 %3, %21
  %22 = call i64 @llvm.vscale.i64()
  %23 = mul i64 %22, 4
  %24 = insertelement <vscale x 4 x float> shufflevector (<vscale x 4 x float> insertelement (<vscale x 4 x float> poison, float -0.000000e+00, i64 0), <vscale x 4 x float> poison, <vscale x 4 x i32> zeroinitializer), float %mul13.i, i32 0
  br label %vector.body

; Absence of store instructions
vector.body:                                      ; preds = %vector.body, %vector.ph
  %index = phi i64 [ 0, %vector.ph ], [ %index.next, %vector.body ]
  %vec.phi = phi <vscale x 4 x float> [ %24, %vector.ph ], [ %31, %vector.body ]
  %25 = add i64 %index, 0
  %26 = getelementptr float, ptr addrspace(1) %8, i64 %25
  %27 = getelementptr float, ptr addrspace(1) %26, i32 0
  %wide.load = load <vscale x 4 x float>, ptr addrspace(1) %27, align 4, !tbaa !15, !alias.scope !17
  %28 = fmul reassoc <vscale x 4 x float> %wide.load, shufflevector (<vscale x 4 x float> insertelement (<vscale x 4 x float> poison, float 1.230000e+02, i64 0), <vscale x 4 x float> poison, <vscale x 4 x i32> zeroinitializer)
  %29 = getelementptr float, ptr addrspace(1) %15, i64 %25
  %30 = getelementptr float, ptr addrspace(1) %29, i32 0
  %wide.load9 = load <vscale x 4 x float>, ptr addrspace(1) %30, align 4, !tbaa !15, !alias.scope !20
  %31 = call reassoc <vscale x 4 x float> @llvm.fmuladd.nxv4f32(<vscale
```

```
    x 4 x float> %28, <vscale x 4 x float> %wide.load9, <vscale x 4 x
    float> %vec.phi)
  %index.next = add nuw i64 %index, %23
  %32 = icmp eq i64 %index.next, %n.vec
  br i1 %32, label %middle.block, label %vector.body, !llvm.loop !22


middle.block:                                     ; preds = %vector.
    body
  %33 = call reassoc float @llvm.vector.reduce.fadd.nxv4f32(float
    -0.000000e+00, <vscale x 4 x float> %31)
  store float %33, ptr addrspace(1) %arrayidx.i, align 4, !tbaa !15
  br label %scalar.ph
```

Listing 17: LLVM IR demonstrating SVE Memory Promotion.

Proof lies within the vector.body block, where the accumulation occurs entirely within the scalable vector register %vec.phi using the the @llvm.fmuladd.nxv4f32 intrinsic alongside with the absence of memory store instructions in this loop.

To accommodate the Polyhedral model, we restructured the code to accumulate the dot product within a temporary scalar register (`sum`) before writing the final result back to main memory, as shown in Listing 18.

```
for(size_t i = 0; i < size_; i++) {
  for(size_t j = 0; j < size_; j++) {
    float sum = ptrC[i * size_ + j] * beta;
    {
      #pragma clang fp reassociate(on)
      #pragma clang loop vectorize_width(4, scalable)
      #pragma clang loop interleave_count(1)
      for(size_t k = 0; k < size_; k++) {
        sum += alpha * ptrA[i * size_ + k] * ptrA[j * size_ + k];
      }
      ptrC[i * size_ + j] = sum;
    }
  }
}
```

Listing 18: Fused SYRK kernel utilizing an explicit local scalar variable for accumulation.

Benchmarking these two structural variations revealed a stark performance inversion. The explicit-scalar formulation (Listing 18) achieved excellent optimization under Polly, but standard SVE auto vectorization failed. Conversely, the direct-memory formulation (Listing 16) triggered SVE vectorization, but caused Polly's execution time to severely degrade.

5.5 Polyhedral Dependence vs SVE Memory Promotion in SYRK

The inverted execution times between the explicit scalar and direct memory version highlight the heuristics between the Polyhedral model and standard LLVM canonicalization

passes.

As detailed in "Polly-ACC Transparent compilation to heterogeneous hardware" surrounding Polly[3], the polyhedral model relies on the mathematical analysis of memory accesses. When memory is updated directly inside the innermost loop (Listing 16), the algorithm utilizes manually linearized 1D array indexing and according to the authors, such linearized expressions force the polyhedral model to over approximate data dependencies, as proving definitive non aliasing between arrays ptrC and ptrA becomes mathematically difficult. Therefore, to ensure correctness Polly conservatively abandons the transformation and falls back to a non-optimized scalar execution.

Conversely, when the accumulation is loaded into an explicit local scalar variable sum (Listing 18), LLVM lowers this variable into a scalar PHI node within the IR and Polly's architecture explicitly supports modeling for inter-basic-block scalar use-def relations as well as PHI nodes allowing it to completely bypass array aliasing constraints. Finally enabling Polly to prove the independence of the reduction loop and tile the iteration space, resulting in an optimized execution time.

6 Results

This section presents the performance results gathered from our evaluation using the Polybench benchmark collection. To ensure the reliability of our measurements, each kernel was executed ten times and the data presented in the analyses represents the median execution time across these iterations.

The problem sizes chosen for each benchmark were adopted from the methodology presented by Faqir-Rhazoui and Carlos García,[4] were they demonstrate SYCL as a viable programming prototype for achieving portable performance in edge computing environments, therefore, arranging our input parameters with their research provides a baseline for our evaluation.

Table 1 details the specific benchmarks selected from the Polybench collection, alongside their functional descriptions and the corresponding input parameter dimensions used throughout our testing phase.

Executing the aforementioned kernels, we obtained the performance metrics illustrated in Figure 1.1 The evaluation measured the median execution time across three discrete compilation paths as listed in Methodology section 3.

[3]Grosser, T., et al. *Polly-ACC: Transparent compilation to heterogeneous hardware.* `https://htor.inf.ethz.ch/publications/img/pollyacc.pdf`

[4]*SYCL in the Edge: Performance and Energy Evaluation for Heterogeneous Acceleration.*`https://d-nb.info/1330115996/34`

Table 1: PolyBench collection subset and corresponding input parameter sizes used for evaluation.

Benchmark	**Problem Size (N)**	**Description**
2DConv	4096	2D Image Convolution
2MM	1024	Two successive Matrix Multiplications
BiCG	16384	BiConjugate Gradient Subroutine
GEMM	1024	General Matrix Multiply
SYRK	1024	Symmetric Rank-k Update

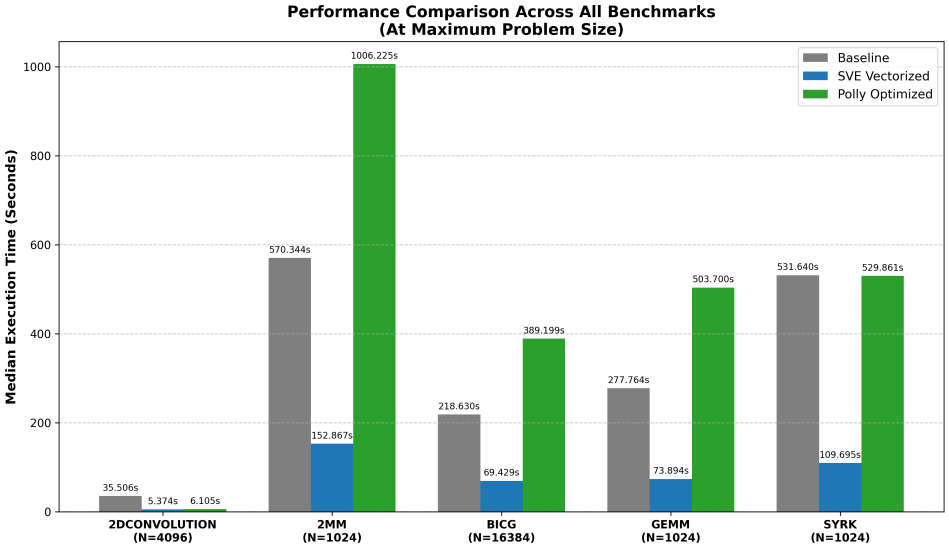

Figure 1: Performance metrics of the evaluated kernels.

6.1 Performance Overview of SVE

Across all problem sizes, the SVE vectorization path perfectly optimized the performance metrics, demonstrating the computational bandwidth of the core when memory access patterns align with hardware vector expectations. Compared to the scalar baseline, the native SVE implementation achieved large time reductions across the entire benchmark suite.

2Dconvolution: The execution time dropped from 35,5ms to 5,57ms.

2MM: The execution time decreased from 570.3ms to 152.8ms.

BiCG: The kernel execution dropped from 218.6ms to 69.4ms.

GEMM: Execution time was reduced from 277.7ms to 73.8ms.

SYRK: The SVE path accelerated the execution from 531.6ms down to 109.6ms. These metrics secured an optimized baseline, proving how the Neoverse-V2 architecture processed contiguous memory streams.

6.2 Performance Overview of Polly

The initial evaluations of the Polly optimized path revealed a failure in optimization across the benchmark suite. For the majority the integration of polyhedral loop transformations resulted degraded performance, doubling the execution times relative to the baseline path.

However, two exceptions emerged for 2DConvolution and SYRK kernels. The 2DConvolution experienced a speedup from 35.5 ms to 6.1ms. In contrast, Polly's static analysis rejected the loop transformation and applied no polyhedral optimizations, resulting a flat performance compared to baseline path.

To determine the reason for this behavior in performance, we used Linux perf subsystem to diagnose the bottleneck.

6.2.1 Polly's Bottleneck Diagnosis

To achieve consistency in micro architecture for the host Cluster Spithas, we inspected the cpu with lscpu. Spithas features a heterogeneous ARMv9 big.LITTLE achritecture containing Cortex-A725 cores (first to tenth) capped at 2808 MHz and Cortex-X925 (eleventh to twentieth) cores able to reach 3900 MHz. To evaluate the optimization paths under maximum resource availability and prevent threads jumping to differnet cores the Linux taskset command was utilized to bind the execution to fifteenth Core, we also executed 10 times every benchmark.

The hardware counters revealed that Polly's degraded performance halts from a structural failure in loop-tiling algorithms, where linear memory assumptions required by the LLVM loop vectorizer for SVE generation forced a fallback to scalar execution generating vast memory read-write instructions leading to long execution unit stalls.

Table 2: Comparison of Polly and SVE Read-Write Performance Paths

Kernel	Optimization	Instructions	Cycles	IPC	L1D Loads	L1D Stores
GEMM	SVE	1.95 B	312.7 M	6.25	542.5 M	276.8 M
	Polly	5.08 B	2.01 B	2.53	1.79 B	1.08 B
2MM	SVE	4.97 B	650.8 M	7.63	1.62 B	547.0 M
	Polly	9.06 B	4.01 B	2.26	3.59 B	2.17 B
BiCG	SVE	27.70 B	13.90 B	1.99	6.42 B	3.48 B
	Polly	30.55 B	14.75 B	2.07	7.19 B	3.69 B

6.2.2 GEMM Pipeline Stavation

Compared with the high speedups achieved from SVE path in the previous section, when polly was applied, the instructions increased almost 2.7 times, multiplied from 1.95 billion to 5.08 billion, while the data cache loads (event 0x40) escalated from 542 million to 1.79 billion, resulting in massive memory requests and index calculation overhead, leading the execution unit to starve, marking the IPC to 2.53.

6.2.3 2MM Write Instruction Bottleneck

The 2MM benchmark experienced a high parallel degradation pattern in Polly path, compared to the SVE path succeed an IPC of 7.63, Polly's multi-dimensional tiling logic forced a fall back to scalar instructions, increasing the amount from 4.96 billion to 9.06 billion. Moreover, L1 cache stores (event 0x41) jumped nearly 4 times from 547 million to 2.16 billion, starving the pipeline store buffers. As a result, the IPC for Polly path decreased to 2.26.

6.2.4 BiCG Memory Bandwidth Bound

Unlike 2MM and GEMM kernels, the BiCG benchmark suffers from a memory bandwidth bound across all compilation paths. The proof lies in the scalar baseline with a backend stall rate of 60.74 percent across 31.3 billion instructions. However, when we applied the SVE vectorization, the compiler successfully reduced the total instructions to 27.7 billion, but the core remained bottlenecked up memory latency with 63.03 percent. Although Polly did not induce new stall cycles, the instructions dropped to 30.5 billion, with an IPC of 2.03 and 62.87 percent backend stall.

6.3 SYRK Optimization Failure

In contrast to the aforementioned kernels, where Polly produced different micro-architectural shifts, the optimizer failed to recognize the computational access patterns within the SYRK benchmark. Consequently, Polly aborted its transformations, generating LLVM IR and ARM machine code identical to baseline path, as mentioned in Section 5.5

7 Conclusions

In this thesis we evaluated a custom end-to-end LLVM compilation pipeline designed to layout the SYCL high abstractions down to the ARM Neoverse-V2 architecture. Initially, by replacing SYCL parallel_for with explicit single_task C++ nested loops, exposed the primary iterations to the LLVM middle-end, allowing us a direct performance comparison between the standard LLVM auto vectorizer and the Polly polyhedral optimization framework. Based on dynamic hardware of Cortex-X925, this research yields several insights regarding compiler optimizations.

7.1 The Efficacy of Native SVE Vectorization

The standared LLVM Loop Vectorizer is capable of generating effecient, vector lenght agnostic SVE instructions, when memory access patterns are linear and contiguous. However, achieving this performance often requires manual code canonicalization, as demonstated in GEMM and SYRK kernels, manual loop interchange and loop fusion were required to remove redundant memory sweeps and allow the backend to utilze SVE fused multiply-addition hardware effectively. Once properly structed, the SVE path yielded massive execution time reductions across the benchmark suite.

7.2 Polyhedral Heuristic Clash

While polyhedral optimization theoretically improve data locality, this evaluation proved that Polly's current heuristics are disconnected from the modern scalable vector hardware, with the exception of the 2DConvolution kernel, where Polly successfully tiled the loops while preserving a contiguous memory layout, allowing the LLVM backend to emit optimized SVE instructions. However, for the remaining kernels, complex multi-dimensional loop tiling broke contiguous memory assumptions required by the LLVM vectorizer, resulting in scalar execution. Furthermore, Polly's static analysis proved to be highly cautious with complex array indexing, forcing the polyhedral model to entirely reject the SYRK kernel to avoid potential memory aliasing.

7.3 Pipeline Starvation and Scalar Fallback Penalty

The micro architectural profiling reveals that a superscalar out-of-order processor like Cortex-X925, has fatal performance trade-offs when vectorization is replaced by scalar instructions. This fallback induced by Polly can cause massive increment in total executed instructions. For compute bound kernels like GEMM and 2MM, this scalar implementation filled the L1 cache controller with billions of isolated read/write instructions, starving the execution pipeline and collapsing the IPC. For memory-bound kernels like BiCG, Polly inflated the exectution time by forcing the CPU to execute billions of address calculations instructions while already bottlenecked waiting for main memory.

7.4 Final Thoughts

Overall, this research concludes that both SVE vectorization path and Polly polyhedral framework require targeted advancements to achieve fully autonomous performance on HPC hardware. The LLVM SVE vectorizer demonstrated exceptional hardware utilization and execution speed, however its heavy reliance on manual code canonicalization highlights a limitation in compilers initial frontend analysis. Conversely, Polly demonstrated genuine optimization potential, as evidenced by the acceleration of the 2DConcolution kernel. The frameworks primary limitation is not its tiling logic, but its overly cautious static analysis.